

WEEK 3 – SELLER AND INVENTORY MANAGEMENT SYSTEM

1. Scenario

The Organic Grocery Store gets different products from different sellers. We need a system to store the seller details, product details, and stock details.

This system helps us to know which seller supplies each product and how much stock is available.

2. Seller Table

The Seller table stores information about the sellers.

It contains the seller ID, seller name, and email address.

```
SQL> SELECT * FROM Seller;
```

SELLER_ID	SELLER_NAME	EMAIL
1	Royal Fresh Mart	royalfresh@gmail.com
2	Daily Harvest Store	dailyharvest@gmail.com
3	Pure Green Foods	puregreen@gmail.com
4	Farm Fresh Hub	farmfreshhub@gmail.com

3. Product Table

The Product table stores information about the products.

It contains the product ID, product name, and seller ID.

```
SQL> SELECT * FROM Product;
```

PRODUCT_ID	PRODUCT_NAME	SELLER_ID
101	Organic Mangoes	1
102	Brown Rice	2
103	Fresh Carrots	3
104	Green Bananas	1

4. Inventory Table

The Inventory table stores the stock details of each product.

It contains the inventory ID, product ID, and stock quantity.

```
SQL> SELECT * FROM Inventory;
```

INVENTORY_ID	PRODUCT_ID	STOCK_QUANTITY
1	101	50
2	102	10
3	103	0
4	104	25

5. Relationships

5.1 Seller and Product

One seller can supply more than one product.

So, the relationship between Seller and Product is **One-to-Many (1)**.

The seller_id is used to connect these two tables.

5.2 Product and Inventory

Each product has one inventory record.

So, the relationship between Product and Inventory is **One-to-One (1:1)**.

The product_id is used to connect these two tables.

6. Constraints

We used different constraints to keep the data correct.

Primary Key: Gives a unique ID to each record.

Foreign Key: Connects two tables.

Unique: Does not allow duplicate values.

Not Null: The value cannot be empty.

Check: Allows only valid values based on a condition.

7. Inserted Values

We inserted data into the Seller, Product, and Inventory tables.

7.1 Seller Values

Four sellers were added to the Seller table.

```
SQL> INSERT INTO Seller VALUES (1, 'Royal Fresh Mart', 'royalfresh@gmail.com');
1 row created.
SQL> INSERT INTO Seller VALUES (2, 'Daily Harvest Store', 'dailyharvest@gmail.com');
1 row created.
SQL> INSERT INTO Seller VALUES (3, 'Pure Green Foods', 'puregreen@gmail.com');
1 row created.
SQL> INSERT INTO Seller VALUES (4, 'Farm Fresh Hub', 'farmfreshhub@gmail.com');
1 row created.
SQL> COMMIT;
```

7.2 Product Values

Four products were added to the Product table.

```
SQL> INSERT INTO Product VALUES (101, 'Organic Mangoes', 1);
1 row created.
SQL> INSERT INTO Product VALUES (102, 'Brown Rice', 2);
1 row created.
SQL> INSERT INTO Product VALUES (103, 'Fresh Carrots', 3);
1 row created.
SQL> INSERT INTO Product VALUES (104, 'Green Bananas', 1);
1 row created.
SQL> COMMIT;
Commit complete.
```

7.3 Inventory Values

Four inventory records were added to the Inventory table.

```
SQL> INSERT INTO Inventory VALUES (1, 101, 50);
1 row created.

SQL> INSERT INTO Inventory VALUES (2, 102, 10);
1 row created.

SQL> INSERT INTO Inventory VALUES (3, 103, 0);
1 row created.

SQL> INSERT INTO Inventory VALUES (4, 104, 25);
1 row created.

SQL> COMMIT;

Commit complete.
```

8. Reports

8.1 Seller-wise Product Report

This report shows how many products each seller supplies.

SELLER_NAME	PRODUCT_COUNT
-----	-----
Royal Fresh Mart	2
Daily Harvest Store	1
Pure Green Foods	1
Farm Fresh Hub	0

8.2 Stock Availability Report

This report shows the stock available for each product.

The stock is shown as:

- Available
- Low Stock
- Out of Stock

PRODUCT_NAME	STOCK_QUANTITY
STOCK_STATUS	
Organic Mangoes Available	50
Brown Rice Low Stock	10
Fresh Carrots Out of Stock	0
Green Bananas Available	25

9. Inventory Analysis

9.1 Total Number of Products

This shows the total number of products in the system.

Result: 4

```
SQL> SELECT COUNT(*) AS total_products FROM Product;

TOTAL_PRODUCTS
-----
              4
```

9.2 Products Out of Stock

This shows how many products have zero stock.

Result: 1

```
SQL> SELECT COUNT(*) AS out_of_stock FROM Inventory WHERE stock_quantity = 0;

OUT_OF_STOCK
-----
            1
```

9.3 Highest-Stocked Product

This shows the product with the highest stock.

Result: Organic Mangoes – 50

```
SQL> SELECT p.product_name, i.stock_quantity FROM Product p JOIN Inventory i ON p.product_id = i.product_id WHERE i.stock_quantity = (SELECT MAX(stock_quantity) FROM Inventory);
```

PRODUCT_NAME	STOCK_QUANTITY
Organic Mangoes	50

9.4 Average Inventory

This shows the average stock quantity of all products.

Result: 21.25

```
SQL> SELECT ROUND(AVG(stock_quantity), 2) AS average_inventory FROM Inventory;
```

AVERAGE_INVENTORY
21.25

10. Conclusion

The Seller and Inventory Management System was successfully created using SQL.

We created three tables: Seller, Product, and Inventory. We also connected the tables using relationships and used different constraints to keep the data correct.

The reports help us easily understand the sellers, products, and available stock.